INF3315 - Privacy Preserving Computation

Assignment 4

Sebastian Mai

DUE: Thursday 19.ovember.2026

1 Task 1:

LO1: explain how linking a de-identified dataset with public data re-identifies people, and measure the risk with k-anonymity and sparse cells. The medical survey of course contains a lot of sensitive information about diabetes. By linking the survey with the pulic information which contains name, pnr, age, zip code etc. using shared attributes such as the age, zip code or county it is possible to associate seemingly anonymous medial records with a named individual. Just removing names alone is therefor not sufficient enought to guarantee privacy as other attributes can also work as identifiable attributes allowing us to re-identify people.

In terms of k-anonymity it measures how many individuals share the same combination of identifiers. So generally sepaking the higher the k the better the anonymity. If $k = 1$ then there is only a single person with the same attributes. This means there is a complete and unique re-identification. In the heatmap all k from 1-4 are marked as sparse cells. A total of 8405 people are uniquely identifiable. 1380 share an attribute and 215 people habe 3 or 4 attributes shared. Noticable is also that County adds no information that isn't already included in the zip code. For example looking at k=1 the combination of zipcode + age identifies 8405 people uniquely. The combination of zipcode, age and county identifies the same amount. If zip code is removed it identification becomes much harder as more people are included in a municipality (counties usually have more than a single zipcode)

When looking at what the API reveals about a person, we must say quite a lot. Taking Andrea Antonsen, 21, 0308 then by calling the API with getDiabeticsByZip(0308) returns 1, meaning there is only a single person that is diabetic in that county. Then calling getAvgAgeByZip(0308) returns 21. So we also get the age of that person. Checking the pii we can get all other information such as the name.

A little bit more about sparse cells. Looking at the county-age heatmap numbers a total of 3360 contained fewer than 5 people. Those cells contained 52% all the entire dataset. Sparse cells pose a privacy risk as fewer people in the combination means also easier indentification.

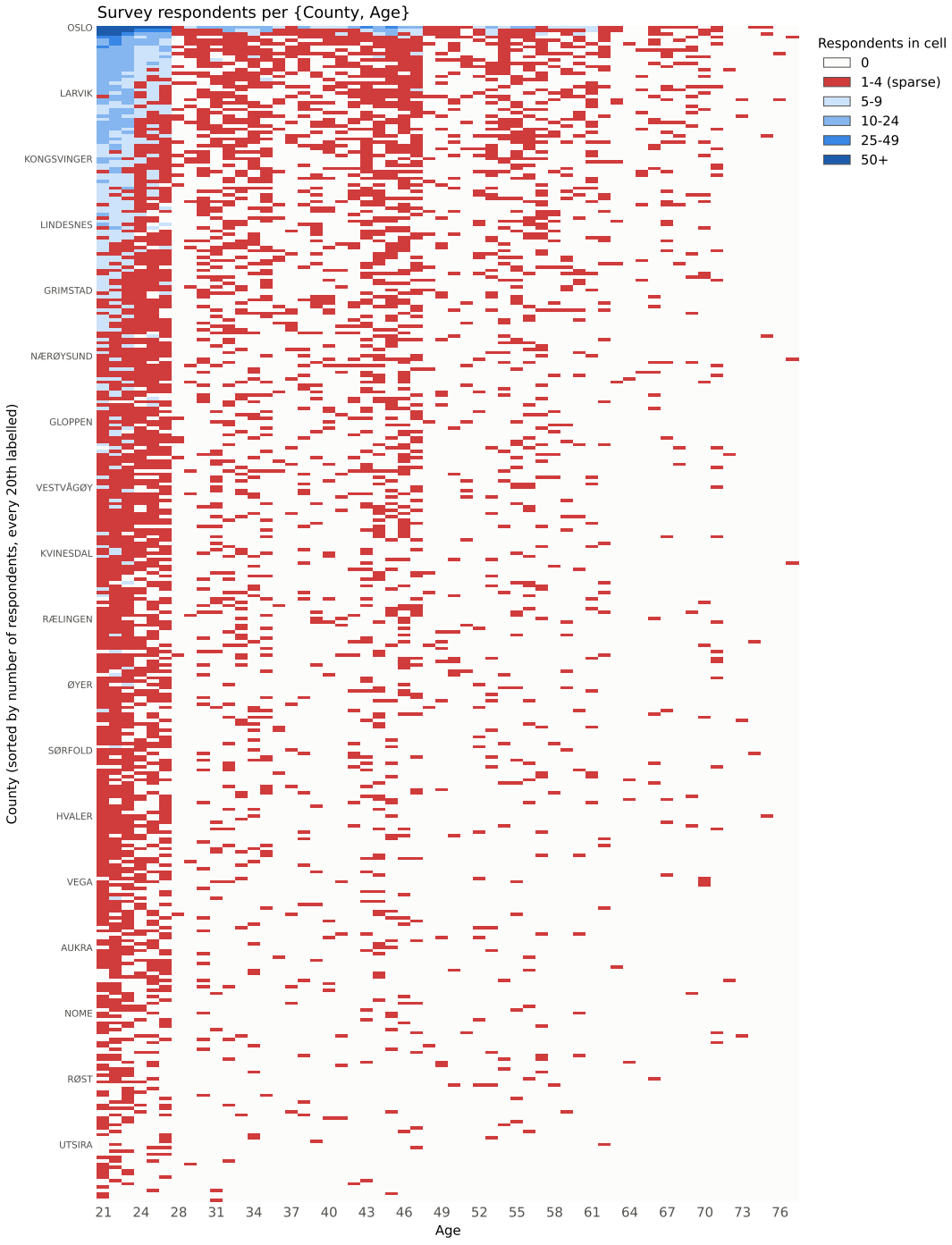


Figure 1: Survey respondents per {County, Age}. Counties are sorted by number of respondents; cells with 1–4 respondents are marked red as sparse.

2 Task 2:

LO2-3: derive the sensitivity of count, sum and average queries from their definition, and explain why clamping bounds must be fixed without looking at the data and explain how ϵ trades accuracy against privacy, and why post-processing a noisy result does not weaken the guarantee.

The sensitivity generally describes how much a queries answer can change when one person's record is added or removed. Therefor the logical one for count is 1. Adding or removing 1 not more than 1 can change in the count. For the sum its clamped to between 0 and 100. The reason it that age has a fixed limit. That limit need to be decided before hand and not be based on the data itself as that would leak information otherwise. 0 to 100 sounds like a reasonable range. The sensitivity of avg can not simply be assumed since it is dependant on both the sum of ages as well as the number of people (for example). Therefore we add noise to the sum using the sum sensitivity and then add noise to the count with the count sensitivity. The avg is then just the division. Epsilon controls the amount of noise aded by the Laplace algorithm. The lower the epsilon the higher the noise that is being added and vice versa. This can be seen in the table. Here the mean average error over 100 DP runs is calculated. As clearly seen, the MAE is significantely higher the lower the epsilon. Since laplacian can result in strange results such as negative counts, decimal numbers for people or so we should adjust them so that no unrealistic answers are returned. Since we are only changing the already noisy result changing this reveals no addditional information about the original data.

Query	Group	Arguments	n	True	$\varepsilon = 0.1$	$\varepsilon = 1$	$\varepsilon = 10$
`getDiabeticsByZip`	large	0475	5	5	6.49	0.91	0
`getDiabeticsByZip`	small	0015	1	1	4.55	0.97	0.02
`getDiabeticsByCounty`	large	OSLO	386	386	9.47	1.06	0.01
`getDiabeticsByCounty`	small	AREMARK	1	1	5.74	0.88	0.02
`getDiabeticsByAge`	large	21	586	586	11.74	1	0.01
`getDiabeticsByAge`	small	49	1	1	7.13	0.78	0
`getDiabeticsByCountyAge`	large	OSLO, 21	68	68	10.05	1.07	0.02
`getDiabeticsByCountyAge`	small	ALSTAHAUG, 22	1	1	6.16	0.79	0.01
`getAverageAge`	all	-	3344	36.23	0.73	0.07	0.01
`getAvgAgeByCounty`	large	OSLO	386	36.43	4.83	0.66	0.06
`getAvgAgeByCounty`	small	AREMARK	1	47	46.3	42.09	16.36
`getAvgAgeByZip`	large	0475	5	29.8	44.39	29.93	4.6
`getAvgAgeByZip`	small	0015	1	21	39.48	34.32	14.61

3 Task 3:

LO4: track a privacy budget with sequential composition, and explain when parallel composition applies.

I chose a privacy budget of 5 and a bin size of 10. Parallel composition is: Because each person affects only one bin, we don't add the privacy costs of all 10 bins. We charge only the maximum cost of one bin. The logical consequence from this ofc is that when I request the same histogram for the same district again then I will request the same data hence disjointness no longer holds and we spend another epsilon of our budget. The reason I chose 10 as a bin width despite the test failing is that the accuracy is better for wider bins. A count + noise will only be done once for the bin but the relative skewing of the result is a lot less for wider bins giving us a better accuracy. This is a balance again that needs to be struck.

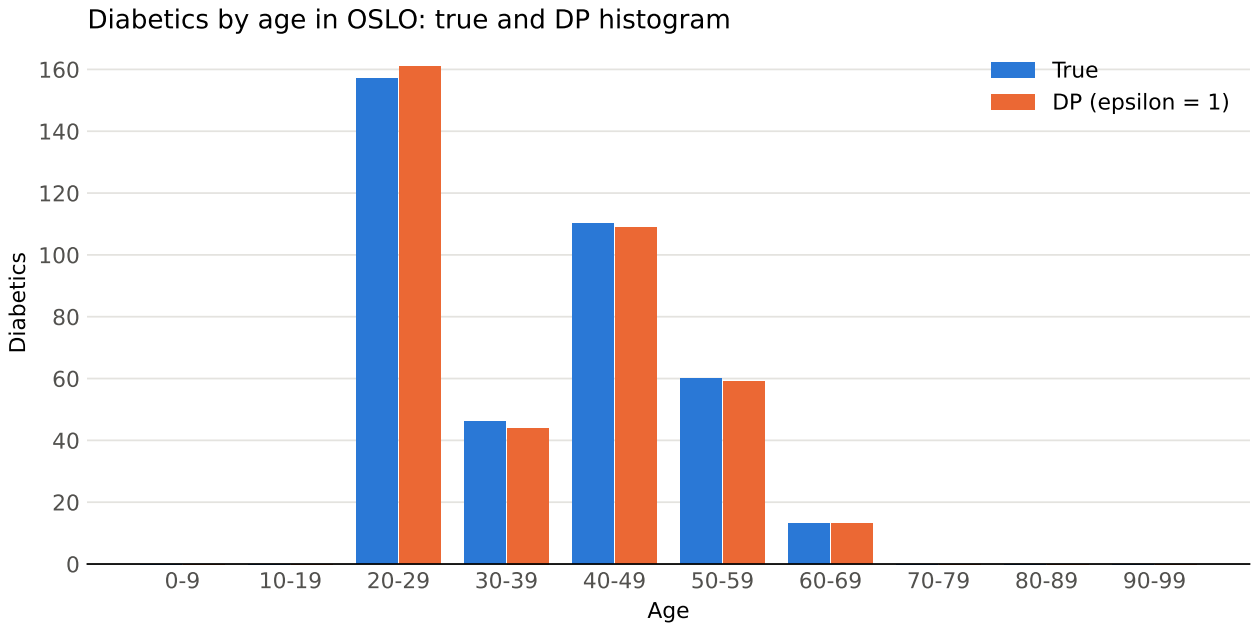


Figure 2: Diabetics by age in Oslo: true histogram (blue) and DP histogram from `getAgeHistogram` (orange), with 10-year age bins and $\epsilon = 1$ charged once for the whole histogram.

4 Task 4:

LO5-6: explain the difference between pure ϵ-differential privacy and (ϵ, δ)-differential privacy, and compute the total privacy loss of repeated Gaussian queries with an accountant and decide, for a given noise level and number of queries, whether Laplace or Gaussian noise gives the lower total privacy loss, and explain why.

Up to 24 calls Laplace is the better choice than Gaussian for the noise level of Task 3. After that the Gaussian has the lower total ϵ'. When it switches depends on the noise level itself. At noise level std 10 Laplace is only better up to 14 calls. So yes the MIT group is correct as Gaussian gives less total ϵ. But this only holds if MIT really does many requests. The reason why Laplace is better at low calls is that it adds it linearly. Gaussian however does not. We calculate the RDP cost based. This is a linear cost. But converting it using the square root is the key part. The square root will grow slower and slower the more queries there are.

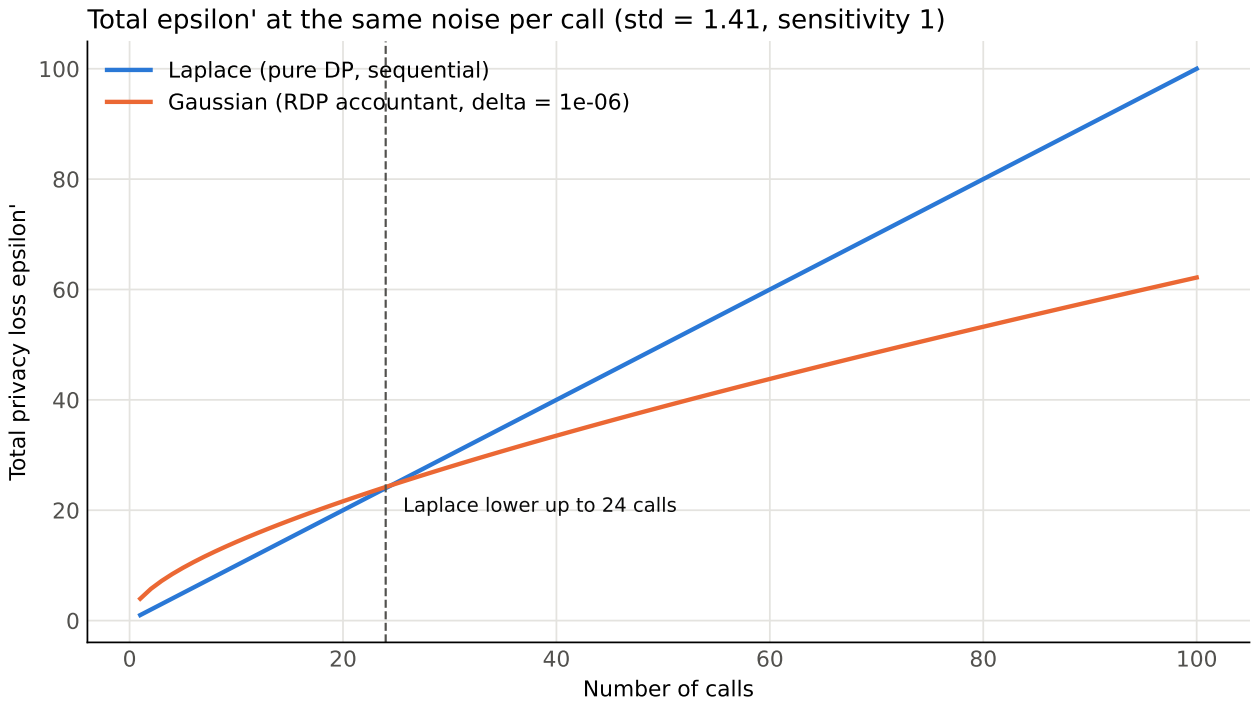


Figure 3: Total privacy loss ϵ' against the number of count queries at the same noise per call (standard deviation $\sqrt{2}$, as in Task 3): Laplace (blue, sequential composition) and Gaussian (orange, RDP accountant at $\delta = 10^{-6}$). Laplace has the lower ϵ' up to 24 calls.

5 AI declaration

The report was not written with AI. The only thing the ai did was adding the figures and tables. The entire code however is written using ai (Level 4) and manually verfied. I used claude code for that. The model doing the work was Opus 5.5. In addition I used the uit-ai-skill. I verfied manually and by running the tests. Most of the answers were correct and little to no change was needed.